

OOP

2.0

Generated by Doxygen 1.9.8

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 Studentas Class Reference	7
4.2 Zmogus Class Reference	7
4.2.1 Detailed Description	8
4.2.2 Constructor & Destructor Documentation	8
4.2.2.1 Zmogus() [1/4]	8
4.2.2.2 Zmogus() [2/4]	8
4.2.2.3 ~Zmogus()	8
4.2.2.4 Zmogus() [3/4]	9
4.2.2.5 Zmogus() [4/4]	9
4.2.3 Member Function Documentation	9
4.2.3.1 getPavarde()	9
4.2.3.2 getVardas()	9
4.2.3.3 operator=() [1/2]	9
4.2.3.4 operator=() [2/2]	10
4.2.3.5 setPavarde()	10
4.2.3.6 setVardas()	10
4.2.3.7 vidurkis()	10
4.2.4 Member Data Documentation	11
4.2.4.1 pavarde_	11
4.2.4.2 vardas_	11
5 File Documentation	13
5.1 include/funkcijos.h File Reference	13
5.1.1 Function Documentation	14
5.1.1.1 atskiriFailai()	14
5.1.1.2 failoGeneravimas()	14
5.1.1.3 failoSpausdinimas()	15
5.1.1.4 KonteineriuTyrimas()	15
5.1.1.5 rikiavimas()	15
5.1.1.6 skaitymas()	16
5.1.1.7 spausdinimas()	16
5.1.1.8 StudentuPadalinimas()	16
5.1.1.9 StudentuPadalinimas2()	17
5.1.1.10 StudentuPadalinimas3()	17

5.1.1.11 tyrimasAntras()	17
5.1.1.12 tyrimasPirmas()	18
5.2 funkcijos.h	18
5.3 include/studentas.h File Reference	22
5.4 studentas.h	22
5.5 include/zmogus.h File Reference	24
5.6 zmogus.h	24
5.7 src/funkcijos.cpp File Reference	25
5.7.1 Function Documentation	25
5.7.1.1 failoGeneravimas()	25
5.7.1.2 failoSpausdinimas()	26
5.7.1.3 spausdinimas()	26
5.7.1.4 tyrimasAntras()	26
5.7.1.5 tyrimasPirmas()	26
5.8 src/main.cpp File Reference	26
5.8.1 Function Documentation	27
5.8.1.1 main()	27
5.8.2 Variable Documentation	27
5.8.2.1 pavardes	27
5.8.2.2 vardai	27
Index	29

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Zmogus	7
Studentas	7

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Studentas	Studento klasė su įvertinimais	7
Zmogus	Abstrakti klasė turit, sauganti vardą ir pavardę	7

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

include/funkcijos.h	13
include/studentas.h	22
include/zmogus.h	24
src/funkcijos.cpp	25
src/main.cpp	26

Chapter 4

Class Documentation

4.1 Studentas Class Reference

Studento klasė su įvertinimais.

```
#include <studentas.h>
```

Inheritance diagram for Studentas:

4.2 Zmogus Class Reference

Abstrakti klasė turit, sauganti vardą ir pavardę

```
#include <zmogus.h>
```

Inheritance diagram for Zmogus:

Public Member Functions

- [Zmogus](#) ()=default
Default konstruktorius.
- [Zmogus](#) (const std::string &v, const std::string &p)
konstruktorius su parametrais
- virtual [~Zmogus](#) ()
Virtualus destruktoriaus.
- [Zmogus](#) (const [Zmogus](#) &a)
Kopijos konstruktorius.
- [Zmogus](#) & [operator=](#) (const [Zmogus](#) &a)
Kopijos priskyrimo operatorius.
- [Zmogus](#) ([Zmogus](#) &&a)
Perkėlimo konstruktorius.
- [Zmogus](#) & [operator=](#) ([Zmogus](#) &&a)
Perkėlimo priskyrimo operatorius.
- virtual double [vidurkis](#) () const =0

Grynai virtualus vidurkio metodas Paveldinčios klasės privalo realizuoti šį metodą

- `const std::string & getVardas () const`

Grąžina vardą

- `const std::string & getPavarde () const`

Grąžina pavardę

- `void setVardas (const std::string &v)`

Nustato vardą

- `void setPavarde (const std::string &p)`

Nustato pavardę

Protected Attributes

- `std::string vardas_`

Žmogaus vardas.

- `std::string pavarde_`

Žmogaus pavardė

4.2.1 Detailed Description

Abstrakti klasė turit, sauganti vardą ir pavardę

4.2.2 Constructor & Destructor Documentation

4.2.2.1 Zmogus() [1/4]

```
Zmogus::Zmogus ( ) [default]
```

Default konstruktorius.

4.2.2.2 Zmogus() [2/4]

```
Zmogus::Zmogus (
    const std::string & v,
    const std::string & p ) [inline]
```

konstruktorius su parametrais

Parameters

<i>v</i>	Vardas
<i>p</i>	Pavardė

4.2.2.3 ~Zmogus()

```
virtual Zmogus::~~Zmogus ( ) [inline], [virtual]
```

Virtualus destruktorius.

4.2.2.4 Zmogus() [3/4]

```
Zmogus::Zmogus (
    const Zmogus & a ) [inline]
```

Kopijos konstruktorius.

Parameters

<i>a</i>	Kopijuojamas objektas
----------	-----------------------

4.2.2.5 Zmogus() [4/4]

```
Zmogus::Zmogus (
    Zmogus && a ) [inline]
```

Perkėlimo konstruktorius.

Parameters

<i>a</i>	Priskiriamas objektas
----------	-----------------------

4.2.3 Member Function Documentation

4.2.3.1 getPavarde()

```
const std::string & Zmogus::getPavarde ( ) const [inline]
```

Grąžina pavardę

4.2.3.2 getVardas()

```
const std::string & Zmogus::getVardas ( ) const [inline]
```

Grąžina vardą

4.2.3.3 operator=() [1/2]

```
Zmogus & Zmogus::operator= (
    const Zmogus & a ) [inline]
```

Kopijos priskyrimo operatorius.

Parameters

<i>a</i>	Priskiriamas objektas
----------	-----------------------

Returns

Nuoroda į objektą

4.2.3.4 operator=() [2/2]

```
Zmogus & Zmogus::operator= (
    Zmogus && a ) [inline]
```

Perkėlimo priskyrimo operatorius.

Parameters

<i>a</i>	Perkeliamas objektas
----------	----------------------

Returns

nuoroda į objektą

4.2.3.5 setPavarde()

```
void Zmogus::setPavarde (
    const std::string & p ) [inline]
```

Nustato pavardę

Parameters

<i>p</i>	Pavardė
----------	---------

4.2.3.6 setVardas()

```
void Zmogus::setVardas (
    const std::string & v ) [inline]
```

Nustato vardą

Parameters

<i>v</i>	Vardas
----------	--------

4.2.3.7 vidurkis()

```
virtual double Zmogus::vidurkis ( ) const [pure virtual]
```

Grynai virtualus vidurkio metodas Paveldinčios klasės privalo realizuoti šį metodą

Implemented in [Studentas](#).

4.2.4 Member Data Documentation

4.2.4.1 pavarde_

`std::string Zmogus::pavarde_` [protected]

Žmogaus pavardė

4.2.4.2 vardas_

`std::string Zmogus::vardas_` [protected]

Žmogaus vardas.

The documentation for this class was generated from the following file:

- [include/zmogus.h](#)

Chapter 5

File Documentation

5.1 include/funkcijos.h File Reference

```
#include "studentas.h"
#include <iostream>
#include <string>
#include <vector>
#include <fstream>
#include <sstream>
#include <stdexcept>
#include <algorithm>
#include <list>
#include <deque>
#include <chrono>
#include <iomanip>
```

Include dependency graph for funkcijos.h: This graph shows which files directly or indirectly include this file:

Functions

- void [spausdinimas](#) (std::vector< [Studentas](#) > &A)
Spausdina studentų sąrašą
- void [failoSpausdinimas](#) (std::vector< [Studentas](#) > &A)
Spausina studentų sąrašą ekrane arba faile.
- void [failoGeneravimas](#) (int studKiekis)
Generuoja studentų failą
- void [tyrimasPirmas](#) ()
Atlieka tyrimą - matuoja failų generavimo laiką
- void [tyrimasAntras](#) (std::vector< [Studentas](#) > &A, std::vector< [Studentas](#) > &vargsai, std::vector< [Studentas](#) > &kietekai)
Atlieka tyrimą - matuoja nuskaitymo ir padalinimo laiką
- template<typename konteineris >
void [skaitymas](#) (konteineris &A, std::string failas)
Nuskaityto duomenis iš failo.
- template<typename konteineris >
void [atskiriFailai](#) (std::string failas, konteineris &vargsai, konteineris &kietekai)
Surašo vargšus ir kietekus į atskirus failus.

- `template<typename konteineris >`
`void rikiavimas (konteineris &A, int kriterijus)`
Rikiuoja studentus pagal pasirinktą kriterijų
- `template<typename konteineris >`
`void StudentuPadalinimas (konteineris &A, konteineris &vargsai, konteineris &kietekai)`
1 strategija - dalina studentus į du atskirus konteinerius
- `template<typename konteineris >`
`void StudentuPadalinimas2 (konteineris &A, konteineris &vargsai)`
2 strategija - rikiuoja ir išima vargšus iš galo
- `template<typename konteineris >`
`void StudentuPadalinimas3 (konteineris &A, konteineris &vargsai)`
3 strategija - naudoja partition padalinimui
- `template<typename konteineris >`
`void KonteineriuTyrimas (konteineris &A, konteineris &vargsai, konteineris &kietekai, int kriterijus, int strategija)`
Atlieka skirtingų konteinerių greičio tyrimą su skirtingomis strategijomis.

5.1.1 Function Documentation

5.1.1.1 atskiriFailai()

```
template<typename konteineris >
void atskiriFailai (
    std::string failas,
    konteineris & vargsai,
    konteineris & kietekai )
```

Surašo vargšus ir kietekus į atskirus failus.

Template Parameters

<i>konteineris</i>	Konteineris (vector, list, deque)
--------------------	-----------------------------------

Parameters

<i>failas</i>	Originalaus failo pavadinimas
<i>vargsai</i>	Vargšų studentų konteineris
<i>kietekai</i>	Kietekų studentų konteineris

5.1.1.2 failoGeneravimas()

```
void failoGeneravimas (
    int studKiekis )
```

Generuoja studentų failą

Parameters

<i>studKiekis</i>	Studentų kiekis
-------------------	-----------------

5.1.1.3 failoSpausdinimas()

```
void failoSpausdinimas (
    std::vector< Studentas > & A )
```

Spausina studentų sąrašą ekrane arba faile.

Parameters

<i>A</i>	Studentų vektorius
----------	--------------------

5.1.1.4 KontaineriuTyrimas()

```
template<typename kontaineris >
void KontaineriuTyrimas (
    kontaineris & A,
    kontaineris & vargsai,
    kontaineris & kietekai,
    int kriterijus,
    int strategija )
```

Atlieka skirtingų kontainerių greičio tyrimą su skirtingomis strategijomis.

Template Parameters

<i>kontaineris</i>	Kontaineris (vector, list, deque)
--------------------	-----------------------------------

Parameters

<i>A</i>	Studentų kontaineris
<i>vargsai</i>	Prastai besimokančių studentų kontaineris
<i>kietekai</i>	Gerai besimokančių studentų kontaineris
<i>kriterijus</i>	Rikiavimo kriterijus (1 - vardas; 2 - pavardė; 3 - galutinis pagal vidurkį; 4 - galutinis pagal medianą)
<i>strategija</i>	Padalinimo strategija (1, 2 arba 3)

5.1.1.5 rikiavimas()

```
template<typename kontaineris >
void rikiavimas (
    kontaineris & A,
    int kriterijus )
```

Rikiuoja studentus pagal pasirinktą kriterijų

Template Parameters

<i>kontaineris</i>	Kontaineris (vector, list, deque)
--------------------	-----------------------------------

Parameters

<i>A</i>	Studentų konteineris
<i>kriterijus</i>	Rikiavimo kriterijus (1 - vardas; 2 - pavardė; 3 - galutinis pagal vidurkį; 4 - galutinis pagal medianą)

5.1.1.6 skaitymas()

```
template<typename konteineris >
void skaitymas (
    konteineris & A,
    std::string failas )
```

Nuskaito duomenis iš failo.

Template Parameters

<i>konteineris</i>	Konteineris (vector, list, deque)
--------------------	-----------------------------------

Parameters

<i>A</i>	Studentų konteineris
<i>failas</i>	Failo pavadinimas

5.1.1.7 spausdinimas()

```
void spausdinimas (
    std::vector< Studentas > & A )
```

Spausdina studentų sąrašą

Parameters

<i>A</i>	Studentų vektorius
----------	--------------------

5.1.1.8 StudentuPadalinimas()

```
template<typename konteineris >
void StudentuPadalinimas (
    konteineris & A,
    konteineris & vargsai,
    konteineris & kietekai )
```

1 strategija - dalina studentus į du atskirus konteinerius

Template Parameters

<i>konteineris</i>	Konteineris (vector, list, deque)
--------------------	-----------------------------------

Parameters

<i>A</i>	Studentų konteineris
<i>vargsai</i>	Prastai besimokančių studentų konteineris
<i>kietekai</i>	Gerau besimokančių studentų konteineris

5.1.1.9 StudentuPadalinimas2()

```
template<typename konteineris >
void StudentuPadalinimas2 (
    konteineris & A,
    konteineris & vargsai )
```

2 strategija - rikiuoja ir išima vargšus iš galo

Template Parameters

<i>konteineris</i>	Konteineris (vector, list, deque)
--------------------	-----------------------------------

Parameters

<i>A</i>	Studentų konteineris
<i>vargsai</i>	Prastai besimokančių studentų konteineris

5.1.1.10 StudentuPadalinimas3()

```
template<typename konteineris >
void StudentuPadalinimas3 (
    konteineris & A,
    konteineris & vargsai )
```

3 strategija - naudoja partition padalinimui

Template Parameters

<i>konteineris</i>	Konteineris (vector, list, deque)
--------------------	-----------------------------------

Parameters

<i>A</i>	Studentų konteineris
<i>vargsai</i>	Prastai besimokančių studentų konteineris

5.1.1.11 tyrimasAntras()

```
void tyrimasAntras (
    std::vector< Studentas > & A,
```

```
std::vector< Studentas > & vargsai,
std::vector< Studentas > & kietekai )
```

Atlieka tyrimą - matuoja nuskaitymo ir padalinimo laiką

Parameters

<i>A</i>	Studentų vektorius
<i>vargsai</i>	Prastai besimokančių studentų vektorius
<i>kietekai</i>	Gerau besimokančių studentų vektorius

5.1.1.12 tyrimasPirmas()

```
void tyrimasPirmas ( )
```

Atlieka tyrimą - matuoja failų generavimo laiką

5.2 funkcijos.h

[Go to the documentation of this file.](#)

```
00001 #ifndef FUNKCIJOS_H
00002 #define FUNKCIJOS_H
00003
00004 #include "studentas.h"
00005 #include <iostream>
00006 #include <string>
00007 #include <vector>
00008 #include <fstream>
00009 #include <sstream>
00010 #include <stdexcept>
00011 #include <algorithm>
00012 #include <list>
00013 #include <deque>
00014 #include <chrono>
00015 #include <iomanip>
00016
00021 void spausdinimas(std::vector<Studentas>& A);
00022
00027 void failoSpausdinimas(std::vector<Studentas>& A);
00028
00033 void failoGeneravimas(int studKiekis);
00034
00038 void tyrimasPirmas();
00039
00046 void tyrimasAntras(std::vector<Studentas>& A, std::vector<Studentas>& vargsai, std::vector<Studentas>&
    kietekai);
00047
00048
00055 template<typename konteineris>
00056 void skaitymas(konteineris& A, std::string failas)
00057 {
00058     std::ifstream input(failas);
00059
00060     if(!input.is_open())
00061     {
00062         throw std::runtime_error("Pasirinkto failo nepavyko atidaryti!");
00063     }
00064     std::string eilute;
00065
00066     getline(input, eilute);
00067
00068     while(getline(input, eilute))
00069     {
00070         std::stringstream ss(eilute);
00071         Studentas s;
00072         std::string vardas, pavarde;
00073         ss » vardas » pavarde;
00074         s.setVardas(vardas);
```

```

00075         s.setPavarde(pavarde);
00076
00077         std::vector<int> pazymiai;
00078         int x;
00079         while(ss >> x)
00080         {
00081             pazymiai.push_back(x);
00082         }
00083
00084         s.setEgzaminas(pazymiai.back());
00085         pazymiai.pop_back();
00086
00087
00088         s.setNd(pazymiai);
00089         s.setGalutinisVid(s.galutinisVid());
00090         s.setGalutinisMed(s.galutinisMed());
00091
00092         A.push_back(s);
00093     }
00094     input.close();
00095 }
00096
00104 template<typename konteineris>
00105 void atskiriFailai(std::string failas, konteineris& vargsai, konteineris& kietekai)
00106 {
00107     std::ofstream vargsuf("vargsai_" + failas);
00108     std::ofstream kietekuf("kietekai_" + failas);
00109
00110     vargsuf << std::left << std::setw(10) << "Vardas" << std::setw(10) << "Pavarde"
00111     << std::setw(20) << "Galutinis(Vid.)" << std::setw(20) << "Galutinis(Med.)" << "\n";
00112
00113     kietekuf << std::left << std::setw(15) << "Vardas" << std::setw(20) << "Pavarde"
00114     << std::setw(20) << "Galutinis(Vid.)" << std::setw(20) << "Galutinis(Med.)" << "\n";
00115
00116     for(auto& s : vargsai)
00117     {
00118         vargsuf << std::setw(15) << s.getVardas() << std::setw(20) << s.getPavarde();
00119
00120         vargsuf << std::setw(20) << std::fixed << std::setprecision(2) << s.getGalutinisVid() <<
00121         std::setw(20) << std::fixed << std::setprecision(2) << s.getGalutinisMed() << "\n";
00122     }
00123
00124     if(!kietekai.empty())
00125     {
00126         for(auto& s : kietekai)
00127         {
00128             kietekuf << std::setw(15) << s.getVardas() << std::setw(20) << s.getPavarde();
00129
00130             kietekuf << std::setw(20) << std::fixed << std::setprecision(2) << s.getGalutinisVid() <<
00131             std::setw(20) << std::fixed << std::setprecision(2) << s.getGalutinisMed() << "\n";
00132         }
00133     }
00134 }
00135
00142 template<typename konteineris>
00143 void rikiavimas(konteineris& A, int kriterijus)
00144 {
00145     auto byVardas = [](const Studentas& a, const Studentas& b)
00146     {
00147         return a.getVardas() < b.getVardas();
00148     };
00149     auto byPavarde = [](const Studentas& a, const Studentas& b)
00150     {
00151         return a.getPavarde() < b.getPavarde();
00152     };
00153     auto byGalVid = [](const Studentas& a, const Studentas& b)
00154     {
00155         return a.getGalutinisVid() < b.getGalutinisVid();
00156     };
00157     auto byGalMed = [](const Studentas& a, const Studentas& b)
00158     {
00159         return a.getGalutinisMed() < b.getGalutinisMed();
00160     };
00161
00162     //Listo rikiavimas
00163     if constexpr (std::is_same_v<konteineris, std::list<Studentas> >)
00164     {
00165         switch(kriterijus)
00166         {
00167             case 1: A.sort(byVardas); break;
00168             case 2: A.sort(byPavarde); break;
00169             case 3: A.sort(byGalVid); break;
00170             case 4: A.sort(byGalMed); break;
00171         }
00172     }

```

```

00173     }
00174     //Vector arba deque rikiavimas
00175     else
00176     {
00177         switch(kriterijus)
00178         {
00179             case 1: std::sort(A.begin(), A.end(), byVardas); break;
00180             case 2: std::sort(A.begin(), A.end(), byPavarde); break;
00181             case 3: std::sort(A.begin(), A.end(), byGalVid); break;
00182             case 4: std::sort(A.begin(), A.end(), byGalMed); break;
00183         }
00184     }
00185
00186 }
00187 }
00188
00196 template<typename konteineris>
00197 void StudentuPadalinimas( konteineris& A, konteineris& vargsai, konteineris& kietekai)
00198 {
00199     for(const auto& s : A)
00200     {
00201         if(s.getGalutinisVid() < 5.0)
00202         {
00203             vargsai.push_back(s);
00204         }
00205         else
00206         {
00207             kietekai.push_back(s);
00208         }
00209     }
00210     A.clear();
00211 }
00212 }
00219 template<typename konteineris>
00220 void StudentuPadalinimas2(konteineris& A, konteineris& vargsai)
00221 {
00222
00223     if constexpr (std::is_same_v<konteineris, std::list<Studentas> >)
00224     {
00225         A.sort([](const Studentas& a, const Studentas& b){
00226             return a.getGalutinisVid() > b.getGalutinisVid();
00227         });
00228     }
00229     else
00230     {
00231         std::sort(A.begin(), A.end(), [](const Studentas& a, const Studentas& b){
00232             return a.getGalutinisVid() > b.getGalutinisVid();
00233         });
00234     }
00235     while(A.back().galutinisVid() < 5)
00236     {
00237         vargsai.push_back(A.back());
00238         A.pop_back();
00239     }
00240     /*
00241     auto it = A.end();
00242
00243     while(it != A.begin())
00244     {
00245         --it;
00246         if(it->getGalutinisVid() < 5)
00247         {
00248             vargsai.push_back(*it);
00249             it = A.erase(it);
00250         }
00251         else{
00252             break;
00253         }
00254     }*/
00255
00256
00257
00258     /*remove_if iskelia konteinerio elementus i jo gala, ties kuriais lambda grazina true
00259     // it yra iteratorius kuri po remove_if rodo i pirma "netikusi" elementa, kuri reikia pasalinti
00260     auto it = remove_if(A.begin(), A.end(), [&vargsai](studentas& a)
00261     {
00262         if(a.galutinisVid < 5.0)
00263         {
00264             vargsai.push_back(a);
00265             return true;
00266         }
00267         return false;
00268     });
00269     //istrinam pagrindinio konteinerio galo elementus, kurie po remove_if buvo ten sudėti
00270     A.erase(it, A.end());
00271     */
00272

```



```

00273 }
00274
00281 template<typename konteineris>
00282 void StudentuPadalinimas3(konteineris& A, konteineris& vargsai)
00283 {
00284     //grazina iteratoriu it, kuris rodo i antro bloko pirma elementa, kuris pagal partition ir musu
    salyga buvo false
00285     //konteineris lieka isdeliotas taip: pirmi elementai tie, kurie tenkina salyga, tada po ju eina
    visi like
00286     auto it = std::partition(A.begin(), A.end(), [](const Studentas& a) { return a.getGalutinisVid()
    >= 5.0; });
00287
00288     //sudeda elementus nuo konteinerio A iteratoriaus iki konteinerio pabaigos i vargsai konteineri
00289     vargsai.insert(vargsai.begin(), it, A.end());
00290     //istrina A konteinerio elementus nuo iteratoriaus iki konteinerio pabaigos
00291     A.erase(it, A.end());
00292 }
00293
00294
00304 template<typename konteineris>
00305 void KonteineriuTyrimas(konteineris& A, konteineris& vargsai, konteineris& kietekai, int kriterijus,
    int strategija)
00306 {
00307     std::vector<int> studKiekis = {1000, 10000, 100000, 1000000, 10000000};
00308
00309     std::cout << std::left << std::setw(12) << "Studentai" << std::setw(15) << "Skaitymas"
00310     << std::setw(15) << "Rikiavimas" << std::setw(15) << "Skirstymas"
00311     << std::setw(15) << "Bendras laikas" << std::endl;
00312
00313
00314     for(auto x : studKiekis)
00315     {
00316         A.clear();
00317         vargsai.clear();
00318         kietekai.clear();
00319
00320         //nuskaitymas
00321         auto start1 = std::chrono::high_resolution_clock::now();
00322         skaitymas(A, "studentai" + std::to_string(x) + ".txt");
00323         auto end1 = std::chrono::high_resolution_clock::now();
00324
00325         std::chrono::duration<double> diff1 = end1 - start1;
00326
00327         //rikiavimas
00328         auto start2 = std::chrono::high_resolution_clock::now();
00329         rikiavimas(A, kriterijus);
00330         auto end2 = std::chrono::high_resolution_clock::now();
00331
00332         std::chrono::duration<double> diff2 = end2 - start2;
00333
00334         //padalinimas
00335         auto start3 = std::chrono::high_resolution_clock::now();
00336         if(strategija == 1)
00337         {
00338             StudentuPadalinimas(A, vargsai, kietekai);
00339         }
00340         else if(strategija == 2)
00341         {
00342             StudentuPadalinimas2(A, vargsai);
00343         }
00344         else if(strategija == 3)
00345         {
00346             StudentuPadalinimas3(A, vargsai);
00347         }
00348         auto end3 = std::chrono::high_resolution_clock::now();
00349
00350         std::chrono::duration<double> diff3 = end3 - start3;
00351
00352         //bendras
00353         double visas = diff1.count() + diff2.count() + diff3.count();
00354
00355         std::cout << std::left << std::setw(12) << x << std::setw(15) << diff1.count()
00356         << std::setw(15) << diff2.count() << std::setw(15) << diff3.count()
00357         << std::setw(15) << visas << std::endl;
00358
00359         atskiriFailai("studentai" + std::to_string(x) + ".txt", vargsai, kietekai);
00360
00361     }
00362     A.clear();
00363     vargsai.clear();
00364     kietekai.clear();
00365 }
00366
00370 #endif

```

5.3 include/studentas.h File Reference

```
#include <string>
#include <vector>
#include <sstream>
#include <iostream>
#include "zmogus.h"
```

Include dependency graph for studentas.h: This graph shows which files directly or indirectly include this file:

Classes

- class [Studentas](#)

Studento klasė su įvertinimais.

5.4 studentas.h

[Go to the documentation of this file.](#)

```
00001 #ifndef STUDENTAS_H
00002 #define STUDENTAS_H
00003
00004 #include <string>
00005 #include <vector>
00006 #include <sstream>
00007 #include <iostream>
00008 #include "zmogus.h"
00009
00018 class Studentas : public Zmogus{
00019
00020 private:
00021     std::vector<int> nd_;
00022     int egzaminas_;
00023     double galutinisVid_;
00024     double galutinisMed_;
00025
00026 public:
00030     Studentas()
00031         : Zmogus(), egzaminas_(0), galutinisVid_(0.0), galutinisMed_(0.0) {}
00032
00038     Studentas(const std::string& v, const std::string& p)
00039         : Zmogus(v, p), egzaminas_(0), galutinisVid_(0.0), galutinisMed_(0.0) {}
00040
00044     ~Studentas() override
00045     {
00046         vardas_.clear();
00047         pavarde_.clear();
00048         nd_.clear();
00049         egzaminas_ = 0;
00050         galutinisVid_ = 0.0;
00051         galutinisMed_ = 0.0;
00052     }
00053
00054
00059     Studentas(const Studentas &a)
00060         : Zmogus(a),
00061           nd_(a.nd_),
00062           egzaminas_(a.egzaminas_),
00063           galutinisVid_(a.galutinisVid_),
00064           galutinisMed_(a.galutinisMed_) {}
00065
00066
00072     Studentas& operator=(const Studentas &a)
00073     {
00074         if(this != &a)
00075         {
00076             Zmogus::operator=(a);
00077             nd_ = a.nd_;
00078             egzaminas_ = a.egzaminas_;
00079             galutinisVid_ = a.galutinisVid_;
00080             galutinisMed_ = a.galutinisMed_;
00081         }
00082         return *this;
00083     }
```

```

00084     }
00085
00090     Studentas (Studentas&& a)
00091     : Zmogus (std::move(a)),
00092       nd_ (std::move(a.nd_)),
00093       egzaminas_ (std::move(a.egzaminas_)),
00094       galutinisVid_ (std::move(a.galutinisVid_)),
00095       galutinisMed_ (std::move(a.galutinisMed_))
00096     {
00097         a.egzaminas_ = 0;
00098         a.galutinisVid_ = 0.0;
00099         a.galutinisMed_ = 0.0;
00100     }
00101
00107     Studentas& operator=(Studentas&& a)
00108     {
00109         if (this != &a)
00110         {
00111             Zmogus::operator=(std::move(a));
00112             nd_ = std::move(a.nd_);
00113             egzaminas_ = std::move(a.egzaminas_);
00114             galutinisVid_ = std::move(a.galutinisVid_);
00115             galutinisMed_ = std::move(a.galutinisMed_);
00116
00117             a.egzaminas_ = 0;
00118             a.galutinisVid_ = 0.0;
00119             a.galutinisMed_ = 0.0;
00120
00121         }
00122
00123         return *this;
00124     }
00125
00132     friend std::ostream& operator<< (std::ostream& out, const Studentas& a)
00133     {
00134         out << a.vardas_ << " " << a.pavarde_ << " ";
00135         for (int i : a.nd_)
00136         {
00137             out << i << " ";
00138         }
00139         out << a.egzaminas_;
00140         return out;
00141     }
00142
00149     friend std::istream& operator>> (std::istream& in, Studentas& a)
00150     {
00151         std::string eilute;
00152         std::getline(in, eilute);
00153         std::stringstream ss(eilute);
00154         ss >> a.vardas_ >> a.pavarde_;
00155
00156         int skaicius;
00157         std::vector<int> visi;
00158
00159         while (ss >> skaicius)
00160         {
00161             visi.push_back(skaicius);
00162         }
00163         a.egzaminas_ = visi.back();
00164         visi.pop_back();
00165         a.nd_ = visi;
00166
00167         return in;
00168     }
00169
00173     const std::vector<int>& getNd() const { return nd_; }
00174
00178     int getEgzaminas() const { return egzaminas_; }
00179
00183     double getGalutinisVid() const { return galutinisVid_; }
00184
00188     double getGalutinisMed() const { return galutinisMed_; }
00189
00194     void setNd(const std::vector<int>& n) { nd_ = n; }
00195
00200     void setEgzaminas(int e) { egzaminas_ = e; }
00201
00206     void setGalutinisVid(double galVid) { galutinisVid_ = galVid; }
00207
00212     void setGalutinisMed(double galMed) { galutinisMed_ = galMed; }
00213
00218     double vidurkis() const override;
00219
00224     double mediana() const;
00225
00230     double galutinisVid() const;
00231

```

```

00236     double galutinisMed() const;
00237
00238
00239 };
00240
00241 #endif

```

5.5 include/zmogus.h File Reference

```

#include <string>
#include <iostream>

```

Include dependency graph for zmogus.h: This graph shows which files directly or indirectly include this file:

Classes

- class [Zmogus](#)

Abstrakti klasė turit, sauganti vardą ir pavardę

5.6 zmogus.h

[Go to the documentation of this file.](#)

```

00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003
00004 #include <string>
00005 #include <iostream>
00006
00012 class Zmogus {
00013 protected:
00014     std::string vardas_;
00015     std::string pavarde_;
00016
00017 public:
00021     Zmogus() = default;
00027     Zmogus(const std::string& v, const std::string& p)
00028         : vardas_(v), pavarde_(p) {}
00029
00033     virtual ~Zmogus()
00034     {
00035         vardas_.clear();
00036         pavarde_.clear();
00037     };
00042     Zmogus(const Zmogus &a)
00043         : vardas_(a.vardas_),
00044           pavarde_(a.pavarde_) {}
00045
00051     Zmogus& operator=(const Zmogus &a)
00052     {
00053         if (this != &a)
00054         {
00055             vardas_ = a.vardas_;
00056             pavarde_ = a.pavarde_;
00057         }
00058         return *this;
00059
00060     }
00061
00066     Zmogus(Zmogus&& a)
00067         : vardas_(std::move(a.vardas_)),
00068           pavarde_(std::move(a.pavarde_)) {}
00069
00075     Zmogus& operator=(Zmogus&& a)
00076     {
00077         if (this != &a)
00078         {
00079             vardas_ = std::move(a.vardas_);
00080             pavarde_ = std::move(a.pavarde_);
00081         }
00082

```

```

00083         return *this;
00084     }
00085
00086
00091     virtual double vidurkis() const = 0;
00092
00093
00097     const std::string& getVardas() const { return vardas_; }
00101     const std::string& getPavarde() const { return pavarde_; }
00102
00107     void setVardas(const std::string& v) { vardas_ = v; }
00112     void setPavarde(const std::string& p) { pavarde_ = p; }
00113
00114 };
00115
00116 #endif

```

5.7 src/funkcijos.cpp File Reference

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <iomanip>
#include <fstream>
#include <chrono>
#include "studentas.h"
#include "funkcijos.h"

```

Include dependency graph for funkcijos.cpp:

Functions

- void [spausdinimas](#) (std::vector< [Studentas](#) > &A)
Spausdina studentų sąrašą
- void [failoSpausdinimas](#) (std::vector< [Studentas](#) > &A)
Spausina studentų sąrašą ekrane arba faile.
- void [failoGeneravimas](#) (int studKiekis)
Generuoja studentų failą
- void [tyrimasPirmas](#) ()
Atlieka tyrimą - matuoja failų generavimo laiką
- void [tyrimasAntras](#) (std::vector< [Studentas](#) > &A, std::vector< [Studentas](#) > &vargsai, std::vector< [Studentas](#) > &kietekai)
Atlieka tyrimą - matuoja nuskaitymo ir padalinimo laiką

5.7.1 Function Documentation

5.7.1.1 failoGeneravimas()

```

void failoGeneravimas (
    int studKiekis )

```

Generuoja studentų failą

Parameters

<i>studKiekis</i>	Studentų kiekis
-------------------	-----------------

5.7.1.2 failoSpausdinimas()

```
void failoSpausdinimas (
    std::vector< Studentas > & A )
```

Spausina studentų sąrašą ekrane arba faile.

Parameters

<i>A</i>	Studentų vektorius
----------	--------------------

5.7.1.3 spausdinimas()

```
void spausdinimas (
    std::vector< Studentas > & A )
```

Spausdina studentų sąrašą

Parameters

<i>A</i>	Studentų vektorius
----------	--------------------

5.7.1.4 tyrimasAntras()

```
void tyrimasAntras (
    std::vector< Studentas > & A,
    std::vector< Studentas > & vargsai,
    std::vector< Studentas > & kietekai )
```

Atlieka tyrimą - matuoja nuskaitymo ir padalinimo laiką

Parameters

<i>A</i>	Studentų vektorius
<i>vargsai</i>	Prastai besimokančių studentų vektorius
<i>kietekai</i>	Gera besimokančių studentų vektorius

5.7.1.5 tyrimasPirmas()

```
void tyrimasPirmas ( )
```

Atlieka tyrimą - matuoja failų generavimo laiką

5.8 src/main.cpp File Reference

```
#include <iostream>
#include <ctime>
```

```
#include <cstdlib>
#include <stdexcept>
#include <list>
#include <deque>
#include "funkcijos.h"
#include "studentas.h"
#include "zmogus.h"
Include dependency graph for main.cpp:
```

Functions

- int [main](#) ()

Variables

- const std::vector< std::string > [vardai](#) = {"Rokas", "Dziugas", "Kajus", "Dovydas", "Matas", "Simonas", "Mantas", "Kasparas", "Tomas", "Kristupas"}
- const std::vector< std::string > [pavardes](#) = {"Brazdeikis", "Kazlauskas", "Macijauskas", "Sabonis", "Valanciunas", "Tubelis", "Sirvydis", "Ulanovas", "Giedraitis", "Kalnietis"}

5.8.1 Function Documentation

5.8.1.1 main()

```
int main ( )
```

5.8.2 Variable Documentation

5.8.2.1 pavardes

```
const std::vector<std::string> pavardes = {"Brazdeikis", "Kazlauskas", "Macijauskas", "Sabonis", "Valanciunas", "Tubelis", "Sirvydis", "Ulanovas", "Giedraitis", "Kalnietis"}
```

5.8.2.2 vardai

```
const std::vector<std::string> vardai = {"Rokas", "Dziugas", "Kajus", "Dovydas", "Matas", "Simonas", "Mantas", "Kasparas", "Tomas", "Kristupas"}
```


Index

- ~Zmogus
 - Zmogus, [8](#)
- atskiriFailai
 - funkcijos.h, [14](#)
- failoGeneravimas
 - funkcijos.cpp, [25](#)
 - funkcijos.h, [14](#)
- failoSpausdinimas
 - funkcijos.cpp, [26](#)
 - funkcijos.h, [15](#)
- funkcijos.cpp
 - failoGeneravimas, [25](#)
 - failoSpausdinimas, [26](#)
 - spausdinimas, [26](#)
 - tyrimasAntras, [26](#)
 - tyrimasPirmas, [26](#)
- funkcijos.h
 - atskiriFailai, [14](#)
 - failoGeneravimas, [14](#)
 - failoSpausdinimas, [15](#)
 - KonteineriuTyrimas, [15](#)
 - rikiavimas, [15](#)
 - skaitymas, [16](#)
 - spausdinimas, [16](#)
 - StudentuPadalinimas, [16](#)
 - StudentuPadalinimas2, [17](#)
 - StudentuPadalinimas3, [17](#)
 - tyrimasAntras, [17](#)
 - tyrimasPirmas, [18](#)
- getPavarde
 - Zmogus, [9](#)
- getVardas
 - Zmogus, [9](#)
- include/funkcijos.h, [13](#), [18](#)
- include/studentas.h, [22](#)
- include/zmogus.h, [24](#)
- KonteineriuTyrimas
 - funkcijos.h, [15](#)
- main
 - main.cpp, [27](#)
- main.cpp
 - main, [27](#)
 - pavardes, [27](#)
 - vardai, [27](#)
- operator=
 - Zmogus, [9](#), [10](#)
- pavarde_
 - Zmogus, [11](#)
- pavardes
 - main.cpp, [27](#)
- rikiavimas
 - funkcijos.h, [15](#)
- setPavarde
 - Zmogus, [10](#)
- setVardas
 - Zmogus, [10](#)
- skaitymas
 - funkcijos.h, [16](#)
- spausdinimas
 - funkcijos.cpp, [26](#)
 - funkcijos.h, [16](#)
- src/funkcijos.cpp, [25](#)
- src/main.cpp, [26](#)
- Studentas, [7](#)
- StudentuPadalinimas
 - funkcijos.h, [16](#)
- StudentuPadalinimas2
 - funkcijos.h, [17](#)
- StudentuPadalinimas3
 - funkcijos.h, [17](#)
- tyrimasAntras
 - funkcijos.cpp, [26](#)
 - funkcijos.h, [17](#)
- tyrimasPirmas
 - funkcijos.cpp, [26](#)
 - funkcijos.h, [18](#)
- vardai
 - main.cpp, [27](#)
- vardas_
 - Zmogus, [11](#)
- vidurkis
 - Zmogus, [10](#)
- Zmogus, [7](#)
 - ~Zmogus, [8](#)
 - getPavarde, [9](#)
 - getVardas, [9](#)
 - operator=, [9](#), [10](#)
 - pavarde_, [11](#)
 - setPavarde, [10](#)

setVardas, [10](#)
vardas_, [11](#)
vidurkis, [10](#)
Zmogus, [8](#), [9](#)